



CS-343: Web Technologies

Dr. Naima Iltaf

Tenured Professor

Naima.iltaf@seecs.edu.pk

Lecture-8

Spring-2026

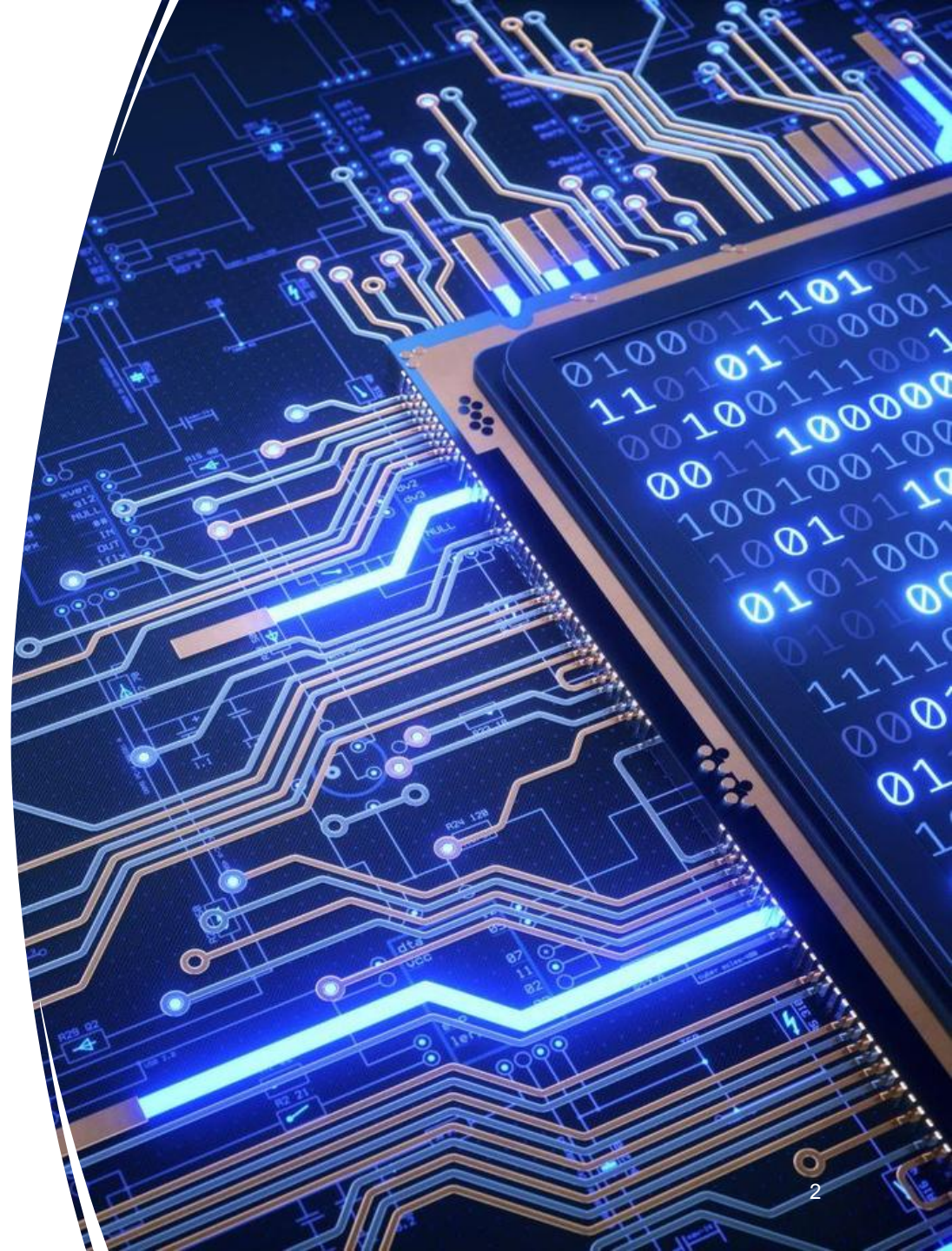
**School of Electrical Engineering and
Computer Science**

**National University of Science and
Technology (NUST)**

Agenda

JavaScript

- Event
- Objects
- JSON



JavaScript: Events

Understanding events

JavaScript code runs sequentially

Events can trigger scripts

Browsers trigger events

Events may happen anytime

JavaScript HTML DOM Events

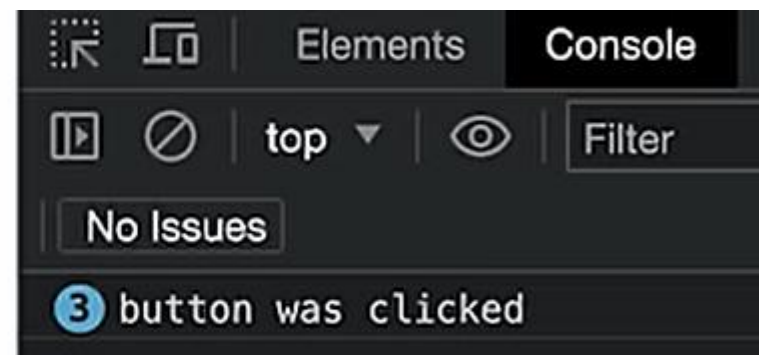
HTML DOM allows JavaScript to react to HTML events

```
<html>
  <head>
    <script>
      function changetext(id){
        id.innerHTML="Oops!";
      }
    </script>
  </head>
  <body>
    <h1 onclick="changetext(this)">Click on this text!</h1>
  </body>
</html>
```

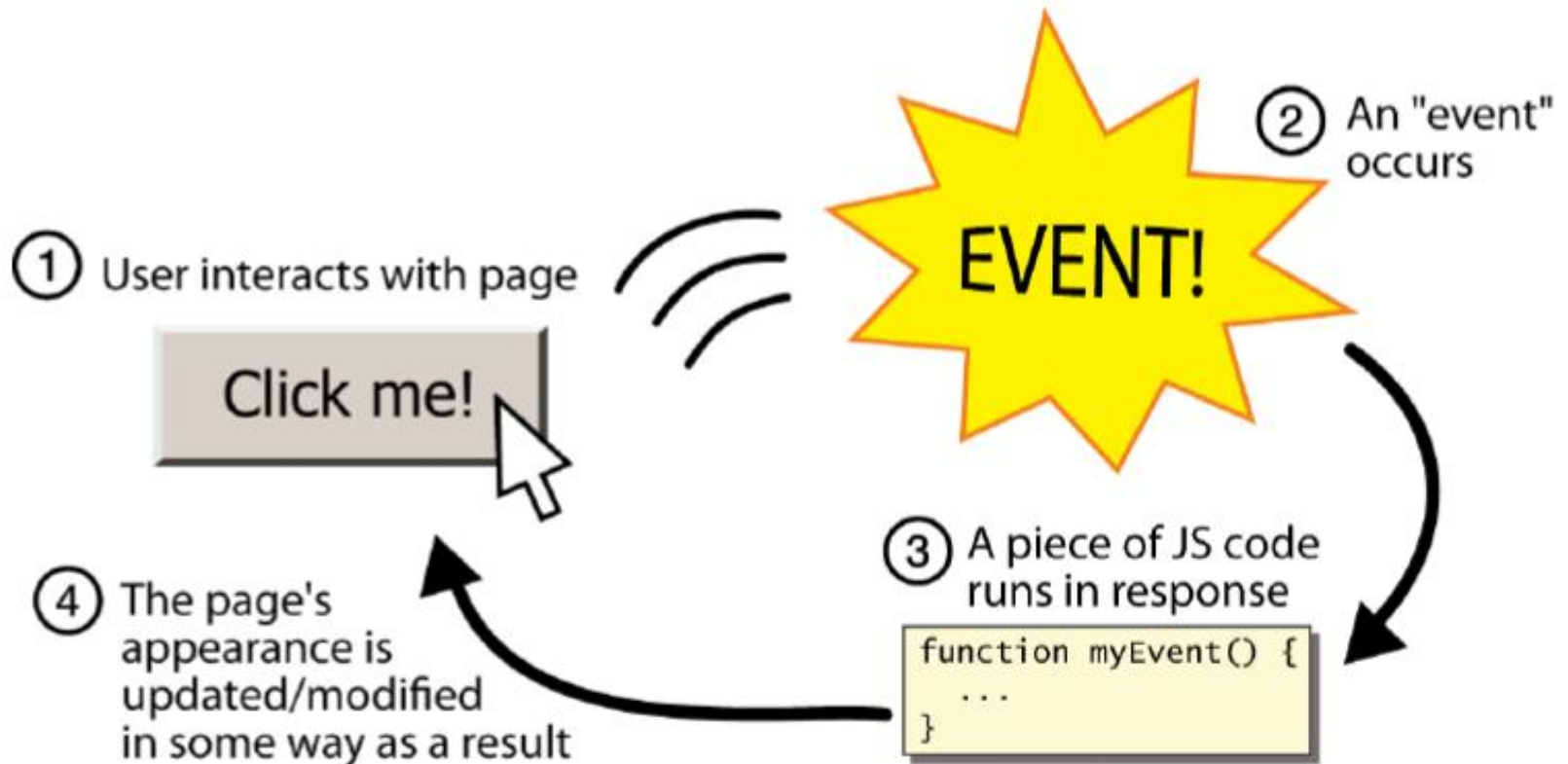
Event: example

```
<body>  
  <button onclick="console.log('button was clicked');">click me!</button>  
  <script src="script.js"></script>  
</body>  
</html>
```

click me!



Event-driven programming



```
<button>Click me!</button>
```

HTML

- button's text appears inside tag; can also contain images
- To make a responsive button or other UI control:
 1. choose the control (e.g. button) and event (e.g. mouse click) of interest
 2. write a JavaScript function to run when the event occurs
 3. attach the function to the event on the control

Event handlers

```
<element attributes onclick="function();">...
```

HTML

```
<button onclick="myFunction();">Click me!</button>
```

HTML

- ❑ JavaScript functions can be set as event handlers
 - when you interact with the element, the function will execute
- ❑ *onclick* is just one of many event HTML attributes
- ❑ but popping up an alert window is disruptive and annoying
 - A better user experience would be to have the message appear on the page...

cc0000

Events & Event Handlers

- Every element on a web page has certain events which can trigger invocation of event handlers
- Attributes are inserted into HTML tags to define events and event handlers
- Examples of events
 - A mouse click
 - A web page or an image loading
 - Mouse hovering over a hot spot on the web page
 - Selecting an input box in an HTML form
 - Submitting an HTML form
 - A keystroke

Events

- `onabort` - Loading of an image is interrupted
- `onblur` - An element loses focus
- `onchange` - The content of a field changes
- `onclick` - Mouse clicks an object
- `ondblclick` - Mouse double-clicks an object
- `onerror` - An error occurs when loading a document or an image
- `onfocus` - An element gets focus
- `onkeydown` - A keyboard key is pressed
- `onkeypress` - A keyboard key is pressed or held down
- `onkeyup` - A keyboard key is released

Events

- onload
 - A page or an image is finished loading
- onmousedown
 - A mouse button is pressed
- onmousemove
 - The mouse is moved
- onmouseout
 - The mouse is moved off an element
- onmouseover
 - The mouse is moved over an element
- onmouseup
 - A mouse button is released
- onreset
 - The reset button is clicked
- onresize
 - A window or frame is resized
- onselect
 - Text is selected
- onsubmit
 - The submit button is clicked
- onunload
 - The user exits the page

onload & Page Exit Events

- The onload event is triggered when a page and its resources are fully loaded
 - It is commonly used to:
 - Initialize JavaScript
 - Fetch data (APIs)
 - Manipulate the DOM
- The onunload event was historically used when leaving a page, but is now deprecated and unreliable
- Modern alternatives:
 - beforeunload (for warnings)
 - visibilitychange / pagehide (for state handling)

Modern web apps don't rely on "user exit" events.
They **proactively save state when visibility changes.**

onFocus, onBlur and onChange

- The *onFocus*, *onBlur* and *onChange* events are often used in combination with validation of form fields.

<u>Event</u>	<u>When it fires</u>	<u>Best Use Case</u>
onfocus	User enters field	Highlight / hints
onblur	User leaves field	Validation / formatting
onChange	Value changed + field lost focus	Final validation
input	Every keystroke	Live validation ★

Example

```
<input type="text" id="fname" onblur="upperCase()">
```

```
function upperCase() {  
  var x = document.getElementById("fname").value;  
  document.getElementById("fname").value = x.toUpperCase();  
}
```

Behavior:

- User types → nothing happens
- User clicks outside → text becomes uppercase

onSubmit

- The *onSubmit* event is used to validate all form fields before submitting it.
- Example: The *checkForm()* function will be called when the user clicks the submit button in the form. If the field values are not accepted, the submit should be canceled.
- The function *checkForm()* returns either *true* or *false*. If it returns true the form will be submitted, otherwise the submit will be cancelled:

```
<form method="post" action="xxx.html" onsubmit="return checkForm()">
```

Example & Demo: onSubmit

```
<html>
<head>
<script type="text/javascript">
    function validate() {
        // return true or false based on validation logic
    }
</script>
</head>

<body>
    <form action="tryjs_submitpage.htm" onSubmit="return validate()">
        Name (max 10 characters): <input type="text" id="fname" size="20"><br />
        Age (from 1 to 100): <input type="text" id="age" size="20"><br />
        E-mail: <input type="text" id="email" size="20"><br />
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

onMouseOver and onMouseOut

- *onMouseOver* and *onMouseOut* are often used to create "animated" buttons.
- Below is an example of an *onMouseOver* event. An alert box appears when an *onMouseOver* event is detected:

```
<a href="http://www.w3schools.com"  
  onmouseover="alert('An onMouseOver event)' </a>
```

Inline Event handling

```
<body>
  <button onclick="console.log('button was clicked');alert('hello!')">
    click me!
  </button>
  <button ondblclick="console.log('button was clicked 2x')">
    click me 2 times!
  </button>
  <div onmouseover="console.log('you are inside div')">this is a box</div>
  <script src="script.js"></script>
</body>
</html>
```

When JavaScript is written directly inside HTML elements, it is called obtrusive event handling

Because:

- JavaScript intrudes into HTML
- Breaks separation of concerns
- Makes HTML messy

Unobtrusive JavaScript

- JavaScript event code seen previously was *obtrusive*, in the HTML; this is bad style
- now we'll see how to write unobtrusive JavaScript code
 - ▣ HTML with minimal JavaScript inside
 - ▣ uses the DOM to attach and execute all JavaScript functions

Unobtrusive JavaScript

- allows separation of web site functionality into:
 - ▣ content (HTML) - what is it?
 - ▣ presentation (CSS) - how does it look?
 - ▣ behavior (JavaScript) - how does it respond to user interaction?

Obtrusive event handlers (bad)

```
<button id="ok" onclick="okayClick();" >OK</button>
```

HTML

```
// called when OK button is clicked  
function okayClick() {  
    alert("booyah");  
}
```

JS

- ❑ this is bad style (HTML is cluttered with JS code)
- ❑ goal: remove all JavaScript code from the HTML body

Attaching an event handler in JavaScript code

```
// where element is a DOM element object  
element.event = function;
```

JS

```
document.getElementById("ok").onclick = okayClick;
```

JS

- it is legal to attach event handlers to elements' DOM objects in your JavaScript code
 - notice that you do not put parentheses after the function's name
- this is better style than attaching them in the HTML
- Where should we put the above code?

When does my code run?

```
<head>
<script src="myfile.js" type="text/javascript"></script>
</head>
<body> ... </body>
```

HTML

```
// global code
var x = 3;
function f(n) { return n + 1; }
function g(n) { return n - 1; }
x = f(x);
```

JS

- your file's JS code runs the moment the browser loads the script tag
 - any variables are declared immediately
 - any functions are declared but not called, unless your global code explicitly calls them

When does my code run?

```
<head>
<script src="myfile.js" type="text/javascript"></script>
</head>
<body> ... </body>
```

HTML

```
// global code
var x = 3;
function f(n) { return n + 1; }
function g(n) { return n - 1; }
x = f(x);
```

JS

- ❑ at this point in time, the browser has not yet read your page's body
 - ❑ none of the DOM objects for tags on the page have been created

A failed attempt to be unobtrusive

```
<head>
<script src="myfile.js" type="text/javascript"></script>
</head>
<body>
<div><button id="ok">OK</button></div>
```

HTML

```
// global code
document.getElementById("ok").onclick = okayClick;
// error: $("ok") is null
```

JS

- ❑ problem: global JS code runs the moment the script is loaded
- ❑ script in head is processed before page's body has loaded
 - no elements are available yet or can be accessed yet via the DOM

The window.onload event

```
// this will run once the page has finished loading
function functionName() {
    element.event = functionName;
    element.event = functionName;
    ...
}
window.onload = functionName; // global code
```

JS

- we want to attach our event handlers right after the page is done loading
 - ▣ there is a global event called `window.onload` event that occurs at that moment (**after the page is loaded**)
- in `window.onload` handler we attach all the other handlers to run when events occur

An unobtrusive event handler

```
<!-- look Ma, no JavaScript! -->  
<button id="ok">OK</button>
```

HTML

```
// called when page loads; sets up event handlers  
function pageLoad() {  
    document.getElementById("ok").onclick = okayClick;  
}  
function okayClick() {  
    alert("booyah");  
}  
window.onload = pageLoad; // global code
```

JS

LIMITATION

```
<button id="btn">Click Me</button>
```

```
<script>
```

```
let btn = document.getElementById("btn");
```

```
btn.onclick = function(){  
  console.log("First handler");  
};
```

```
btn.onclick = function() {  
  console.log("Second handler");  
};
```

```
</script>
```

 Output when clicked:

Second handler

The first function is completely lost / overwritten

Define Event Handler

- first choose an html element to add it to
- then call the `addEventListener()` method on the target element with the syntax

```
targetElement.addEventListener(type, listener [, useCapture])  
targetElement.addEventListener(type, listener [, options])
```

- **type** (type of event, such as “click”, “submit”, or “keypress)
- **listener** is a function that handle's what happens when the event is triggered. This can be either a **callback function**
- the third is optional.

```
targetElement.addEventListener("click", sayHello)  
function sayHello() {  
    alert("Hello World!")  
}
```

Example

```
targetElement.addEventListener("click", sayHello)
function sayHello() {
  alert("Hello World!")
}
```

- An anonymous function

```
targetElement.addEventListener("click", function() {
  alert("Hello World!")
})
```

- An arrow function

```
targetElement.addEventListener("click", () => alert("Hello World!"))
```


addEventListener() adds handlers to a list (queue-like behavior)

```
<< HTML
```

```
<button id="btn">Click Me</button>
```

```
<script>
```

```
let btn = document.getElementById("btn");
```

```
// First handler
```

```
btn.addEventListener("click", function() {  
    console.log("Handler 1");  
});
```

```
// Second handler
```

```
btn.addEventListener("click", function() {  
    console.log("Handler 2");  
});
```

```
// Third handler
```

```
btn.addEventListener("click", function() {  
    console.log("Handler 3");  
});  
</script>
```

When Should My JavaScript Run?

? Problem

- JavaScript may run before HTML elements are loaded
- This causes errors like:

```
document.getElementById("btn") → null ✗
```

◆ Solution 1: window.onload

```
window.onload = function() {  
    // runs after FULL page loads (HTML + images + CSS)  
};
```

✓ Works

! Slow (waits for everything)

When Should My JavaScript Run?

- ◆ Solution 2: `DOMContentLoaded`  (Recommended)

```
document.addEventListener("DOMContentLoaded", function() {  
    // runs when DOM is ready (HTML loaded)  
});
```

- ✓ Faster
- ✓ Ideal for event handling

👉 Fires when:

- HTML parsed 
- DOM created 
-  DOES NOT wait for images/CSS

Best Practices

```
<button id="ok">OK</button>
```

```
<script>
```

```
document.addEventListener("DOMContentLoaded", function() {  
    document.getElementById("ok").addEventListener("click",  
function() {  
    alert("booyah");  
    });  
});  
</script>
```

Case 1: Inline + .onclick = (DOM property)

- If you define an event **inline AND in JavaScript**

```
</> HTML
```

```
<button id="btn" onclick="inlineFunc()">Click</button>
```

```
</> JavaScript
```

```
document.getElementById("btn").onclick = jsFunc;
```

`.onclick` = **overwrites** the existing handler (including inline)

Case 2: Inline + addEventListener()

`</>` HTML

```
<button id="btn" onclick="inlineFunc()">Click</button>
```

`</>` JavaScript

```
document.getElementById("btn").addEventListener("click", jsFunc);
```

👉 Both run

Execution order:

1. Inline handler runs first
2. `addEventListener` handler runs after

Practice Question

`</>` HTML

```
<button id="btn" onclick="console.log('Inline')">Click</button>
```

```
<script>
```

```
document.getElementById("btn").addEventListener("click", function() {  
    console.log("JS Listener");
```

```
});
```

```
</script>
```

Practice Question

`</>` HTML

```
<button id="btn" onclick="console.log('Inline')">Click</button>
```

```
<script>
```

```
document.getElementById("btn").addEventListener("click", function() {  
    console.log("JS Listener");
```

```
});
```

```
</script>
```

 Output:

Inline

JS Listener

Summary

Combination	What happens
Inline + .onclick =	Inline is overwritten
Inline + addEventListener()	Both run
Multiple .onclick =	Only last one runs
Multiple addEventListener()	All run

JavaScript Object

- JavaScript is an Object Oriented Programming (OOP) language. It supports programming with objects.
- **Objects are a way of organizing the variables.**
- The different screen elements such as Web pages, forms, text boxes, images, and buttons are treated as objects.
- Objects are just data, with properties and methods
- **Properties** are **values** associated with objects.
- **Methods** are **actions** that objects can perform.

Objects - Properties and Methods

- **Objects** are **variables** that can store both **values** and **functions**.
- **Values** are stored as **key:value** pairs called **properties**.
- **Functions** are stored as **key:function()** pairs called **methods**.

```
const car = {  
  type: "Fiat",  
  model: "500",  
  color: "white"  
};
```

`type`, `model`, and `color` are **properties**

`"Fiat"`, `"500"`, and `"white"` are **property values**

Object Methods

- Methods are **actions** that can be performed on objects.
- Methods are **stored in properties** as **function definitions**.

```
var person = {  
  firstName : "John",  
  lastName  : "Doe",  
  id        : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

Accessing Object Properties and Methods

- Two ways to access object properties
 - ***objectName.propertyName*** e.g. car.color
 - ***objectName["propertyName"]*** e.g. car["color"]
- Accessing object methods
 - ***objectName.methodName()***
 - e.g. **person.fullName();**
- *If you access the fullName method, without (), it will return the function definition:*
 - **name = person.fullName;**

An object method is a function definition, stored as a property value.

If you access it without (), it will return the function definition:

```
function () { return this.firstName + " " + this.lastName; }
```

How to Create a JavaScript Object

- An **object literal** is the simplest and most common way to define a JavaScript object

OOJS: Example 1

```
<p id="demo"></p>
<script>
var person = {
    firstName : "John",
    lastName  : "Doe",
    age       : 50,
    eyeColor  : "blue"
};
document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

```
const person = {
  fname:"John",
  lname:"Doe",
  age:25};

let text = "";
for (let x in person) {
  text += person[x];
}
```

Creating an Empty Object

```
<!DOCTYPE html>
<html>
<body>
<h1>Creating JavaScript Objects</h1>
<h2>Creating an Empty Object</h2>

<p id="demo"></p>

<script>
// Create an empty Object
const person = {};

// Add Properties
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";

// Display Data from Object
document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

OOJS: Example 2

```
<p id="demo"></p>
```

```
<script>
```

```
var person = {  
  firstName: "John",  
  lastName : "Doe",  
  id       : 5566  
};
```

```
document.getElementById("demo").innerHTML =  
person["firstName"] + " " + person["lastName"];  
</script>
```

```
const person = {  
  fname:"John",  
  lname:"Doe",  
  age:25};
```

delete person.lname;

```
let text = "";  
for (let x in person) {  
  text += person[x];  
}
```


OOJS: Example 3

```
<p id="demo"></p>
<script>
var person = {
  firstName: "John",
  lastName : "Doe",
  id        : 5566,
  fullName  : function() {
    return this.firstName + " " + this.lastName;
  }
};
document.getElementById("demo").innerHTML =
person.fullName();
</script>
```

```
const person = {
  fname:"John",
  lname:"Doe",
  age:25};
```

delete person["age"];

```
let text = "";
for (let x in person) {
  text += person[x];
}
```

JavaScript Object Properties

```
// script.js
const person = {
  firstname: "John",
  lastname: "Doe",
  age: 50,
};
```

Property Access Using Dots

```
document.getElementById("demo").innerHTML = person.firstname + " is " + person.age;
```

Property Access Using Brackets []

```
document.getElementById("demo").innerHTML = person["firstname"] + " is " + person["age"];
```

```
let n1 = "firstName";
let n2 = "lastName";
let name = person[n1] + " " + person[n2];
document.getElementById("demo").innerHTML = name;
```

OOJS: Example 4

```
<!DOCTYPE html> <html> <body>
<h1>JavaScript Objects</h1>
<p id="demo"></p>
<script>
const myObj = {
  name: "John",
  age: 30,
  myCars: {
    car1: "Ford",
    car2: "BMW",
    car3: "Fiat"
  }
}
let p1 = "myCars";
let p2 = "car2";
document.getElementById("demo").innerHTML = myObj[p1][p2];
</script>    </body> </html>
```



Prototype Inheritance

- All JavaScript objects **inherit** properties and methods **from a prototype**:
- **Date** objects inherit from **Date.prototype**
- **Array objects** inherit from **Array.prototype**
- The **Object.prototype** is on the **top** of the prototype inheritance chain.
- Date objects, Array objects, and all other objects inherit from Object.prototype.

Prototypes in JS

```
</> JavaScript
```

```
const person = {  
  name: "John"  
};
```

```
</> JavaScript
```

```
person.toString()
```

👉 Works! But you didn't define it.

Because it comes from prototype

Date objects inherit from Date.prototype

```
const d = new Date();
```

- JavaScript does **NOT** create a standalone object.
- Instead, it creates a **chain**:

d → Date.prototype → Object.prototype → null

```
const d = new Date();  
  
console.log(d.getFullYear()); // 2026  
console.log(d.getMonth());    // 0-11
```

Prototypes in JS

- A JavaScript object is an entity having state and behavior (properties and method).
- Every JavaScript object has an internal `__proto__` property that points to its prototype, allowing it to inherit properties and methods
- We can set prototype using `__proto__`

If object & prototype have same method, object's method will be used.

Prototypes in JS

- A JavaScript object is an entity having state and behavior (properties and method).
- Object literal creates a single object
- It automatically inherits from **Object.prototype**
- It does **NOT** have prototype
- It uses **__proto__ internally**
- You can modify or change prototype

JavaScript

```
const person = {  
  name: "Ali",           // state (property)  
  age: 20,               // state  
  greet() {              // behavior (method)  
    console.log("Hello");  
  }  
};
```

```
console.log(person.prototype); // undefined
```

```
console.log(person.__proto__);
```

-----> Points to Object.prototype

```
person.toString();  
person.hasOwnProperty("name");
```

From Object.prototype, it inherits `person.valueOf()`;

Add new method directly

```
person.sayBye = function () {  
  console.log("Bye");  
};  
person.sayBye(); // Bye
```

Add method via prototype

```
person.__proto__.sayHi = function () {  
  console.log("Hi from prototype");  
};
```

Modified Object.prototype

Inherit from another object

```
const person = {  
  name: "Ali",      // state  
  age: 20,          // state  
  greet() {         // behavior  
    console.log("Hello");  
  }  
};
```

```
const student = {  
  study() {  
    return "I am studying";  
  }  
}
```

// Student inherits from Person

```
student.__proto__ = person;
```

```
console.log(student.study());
```

```
console.log(student.greet());
```

student



person



Object.prototype

// own method

// inherited

Prototypes in JS

```
const person = {  
  greet() {  
    return "Hello, I am a person";  
  }  
};  
  
const student = {  
  study() {  
    return "I am studying";  
  }  
};
```

```
Object.setPrototypeOf(student, person);
```

```
console.log(student.study()); // own method  
console.log(student.greet()); // from prototype
```

Prototype is a fallback object from which JavaScript inherits properties and methods

```
I am studying  
Hello, I am a person
```

Prototype allows one object to reuse properties and methods of another object

Prototypes in JS: Method Overriding

```
const animal = {  
  speak() {  
    return "Animal sound";  
  }  
};  
  
const dog = {  
  speak() {  
    return "Bark";  
  }  
};  
  
Object.setPrototypeOf(dog, animal);  
  
console.log(dog.speak());
```

 `dog.speak()`

👉 JavaScript checks object first:

`dog.speak()` → found in dog → STOP

It never goes to prototype.

Classes in JS

- Class is a program-code template for creating objects
- Those objects will have some state (variables) & some behaviour (functions) inside it.

```
class MyClass {  
  
    constructor() { ... }  
  
    myMethod() { ... }  
  
}  
  
let myObj = new MyClass();
```

Example

```
class ToyotaCar {  
  start() {  
    console.log("start");  
  }  
  
  stop() {  
    console.log("stop");  
  }  
}  
  
const car1 = new ToyotaCar();  
  
car1.start();  
car1.stop();
```

Example

```
class ToyotaCar {  
  start() {  
    console.log("start");  
  }  
  
  stop() {  
    console.log("stop");  
  }  
  
  setBrand(brand) {  
    this.brand = brand;  
  }  
}
```

```
let fortuner = new ToyotaCar();  
fortuner.setBrand("fortuner");
```

```
let lexus = new ToyotaCar();  
lexus.setBrand("lexus");
```

```
> lexus  
< ▼ ToyotaCar {brandName: 'lexus'} ⓘ  
  brandName: "lexus"  
  ► [[Prototype]]: Object  
>
```

Constructor in JS

“A constructor is a special method inside a class that runs automatically when an object is created

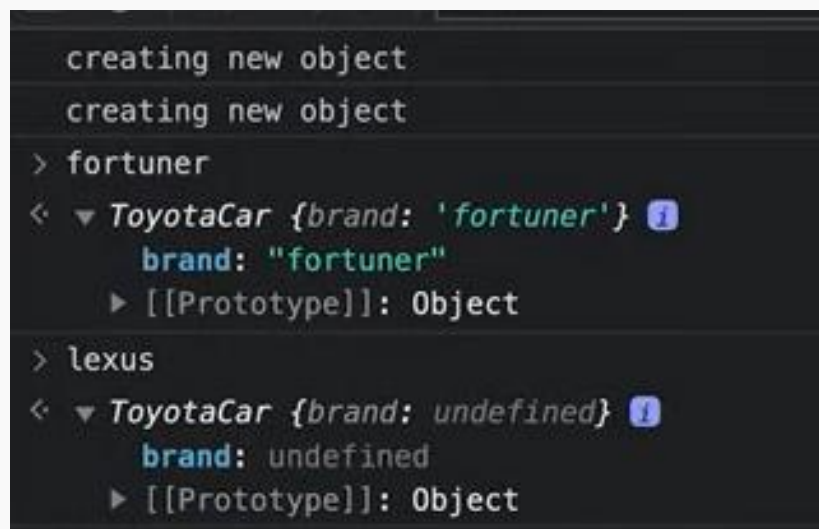
- **Constructor() method is:**
- automatically invoked by new
- initializes object

```
class MyClass {  
  constructor() { ... }  
  myMethod() { ... }  
}
```



```
class ToyotaCar {  
  constructor(brand) {  
    console.log("creating new object");  
    this.brand = brand;  
  }  
  
  start() {  
    console.log("start");  
  }  
  
  stop() {  
    console.log("stop");  
  }  
}
```

```
let fortuner = new ToyotaCar("fortuner"); // constructor  
let lexus = new ToyotaCar("lexus"); // constructor
```



```
class ToyotaCar {  
  constructor(brand) {  
    console.log("creating new object");  
    this.brand = brand;  
  }  
  
  start() {  
    console.log("start");  
  }  
  
  stop() {  
    console.log("stop");  
  }  
}
```

Do you think start() is copied inside both objects?

```
let fortuner = new ToyotaCar("fortuner"); // constructor  
let lexus = new ToyotaCar("lexus"); // constructor
```

Class methods are stored in prototype, not inside objects

```
class ToyotaCar {  
  constructor(brand) {  
    console.log("creating new object");  
    this.brand = brand;  
  }
```

```
  start() {  
    console.log("start");  
  }
```

fortuner → ToyotaCar.prototype → Object.prototype

```
  stop() {  
    console.log("stop");  
  }  
}
```

```
let fortuner = new ToyotaCar("fortuner"); // constructor  
let lexus = new ToyotaCar("lexus"); // constructor
```

```
console.log(fortuner.hasOwnProperty("start")); // false
```

Objects store data, but share methods through prototype

```
class ToyotaCar {  
  start() {}  
}
```

- . Behind the scenes:

```
ToyotaCar.prototype.start = function () {};
```

Class methods are stored in prototype, not inside objects

```
class Car {  
  constructor(brand) {  
    this.brand = brand;  
  }  
  
  start() {  
    console.log("start");  
  }  
}  
  
const c1 = new Car("Toyota");  
  
Car.prototype.stop = function () {  
  console.log("stop");  
};
```

```
c1  
  brand: "Toyota"  
  
↓  
Car.prototype  
  start()  
  stop()  
  
↓  
Object.prototype
```

Controlling Object Mutability in JavaScript

- `Object.preventExtensions()`
- `Object.seal()`
- `Object.freeze()`

They work the **same way on any object**, whether the object was created by:

- object literal
- constructor function
- class

The difference is **not** literal vs class.

Prevent Extensions to Objects

The `Object.preventExtensions()` method prevents adding properties to an object.

You can use `Object.isExtensible()` to check if an object is extensible.

```
<p id="demo"></p>
```

```
<script>
```

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
Object.preventExtensions(fruits);
```

```
// Test Error
```

```
let text = "";
```

```
try {
```

```
  fruits.push("Kiwi");
```

```
}
```

```
catch (err) {
```

```
  text = err;
```

```
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

TypeError: can't define property 4: Array is not extensible

After this:

- you cannot add new properties
- you can still update existing properties
- you can still delete existing properties

Object.preventExtensions(obj)

- Prevents adding new properties to an object
- ✓ Works on:
 - Objects {}
 - Arrays []
 - Class objects

```
class Car {  
  constructor(brand) {  
    this.brand = brand;  
  }  
}  
  
let car1 = new Car("Fortuner");  
  
Object.preventExtensions(car1);  
  
car1.color = "black";    // ✗ Not allowed  
car1.brand = "Corolla"; // ✓ Allowed
```


Freeze Objects

<p id="demo"></p>

<script>

"use strict"

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};
```

// Freeze Object

Object.freeze(person)

// Test Error

let text;

```
try {
  person.age = 51;
  text = Object.values(person);
}
```

```
catch (err) {
```

```
  text = err;
```

```
}
```

document.getElementById("demo").innerHTML = text;

</script>

The `Object.freeze()` method prevents any changes to an object.

The `Object.freeze()` method will fail silently in non-strict mode.

The `Object.freeze()` method will throw a `TypeError` in strict mode.

Frozen objects are read-only. No modification, addition or deletion of properties are allowed.

The `Object.isFrozen()` method can be used to check if an object is frozen.

TypeError: "age" is read-only

After this:

- you cannot add new properties
- you cannot delete properties
- you cannot change existing property values

Seal Objects

The `Object.seal()` method prevents additions or deletions of new properties.

The `Object.seal()` method makes existing properties non-configurable.

The `Object.seal()` method will fail silently in non-strict mode.

The `Object.seal()` method will throw a `TypeError` in strict mode.

The `Object.isSealed()` method can be used to check if an object is sealed.

```
<p id="demo"></p>
```

```
<script>
```

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
Object.seal(fruits);
```

```
// Test Error
```

```
let text = "";
```

```
try {
```

```
  fruits.push("Kiwi");
```

```
}
```

```
catch (err) {
```

```
  text = err;
```

```
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

TypeError: can't define property 4: Array is not extensible

After this:

- you cannot add new properties
- you cannot delete existing properties
- you can update existing properties

Seal vs freeze

Both **Object.seal()** and **Object.freeze()** are used to restrict modifications to JavaScript objects, but they work at different levels.

Feature	Object.seal()	Object.freeze()
Prevents adding new properties?	✓	✓
Prevents deleting existing properties?	✓	✓
Prevents modifying existing properties?	✗ (modification allowed)	✓ (completely read-only)
Extensible (Object.isExtensible())?	✗	✗
Prevents changing property attributes (like writable, configurable)?	✗	✓

Inheritance in JavaScript

extends is just a cleaner way of setting up prototype inheritance

```
class User {  
  greet() {  
    console.log("Hello");  
  }  
}  
  
class Admin extends User {  
  edit() {  
    console.log("Edit data");  
  }  
}  
  
const a1 = new Admin();
```

```
a1  
  ↓  
Admin.prototype  
  |— edit()  
  ↓  
User.prototype  
  |— greet()  
  ↓  
Object.prototype
```

- **super ()** handles constructor inheritance
- **Prototype** handles method inheritance

Practice Example

- You are creating a website for your college. Create a class User with 2 properties, name and email. It also has a method called viewData() that allows user to view website data.
- Create a new class called Admin which inherits from User. Add a new method called editData to Admin that allows it to edit website data.

Practice Example- solution

```
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
  
  viewData() {  
    console.log("Viewing website data");  
  }  
}
```

```
class Admin extends User {  
  constructor(name, email) {  
    super(name, email); // call parent constructor  
  }  
  
  editData() {  
    console.log("Editing website data");  
  }  
}
```

If we want to send this object to a server or save it in a file, can we send it directly?

```
const person = {  
  name: "Ali", age: 20  
};
```

JSON = JavaScript Object Notation

- Lightweight **data-interchange** format
- Represents data in **key–value pairs (text format)**
- **Language-independent** (not limited to JavaScript)
- Easy to **read and write**
- Easy to **parse and generate**
- Widely used in:
 - Web APIs
 - Databases
 - Frontend–backend communication
- Used with:
 - **AJAX and Fetch API for server communication**
 - Based on JavaScript object syntax (**but not full JavaScript**)

JavaScript Object Notation

Object = JavaScript structure

JSON = **string format of that object**

```
const person = {  
  name: "Ali",  
  age: 20  
};
```

```
↪ JSON  
  
{  
  "name": "Ali",  
  "age": 20  
}
```

JavaScript Object Notation

- A collection of **key–value pairs (text format)**
- Keys must be in **double quotes**
- Values separated by :
- Pairs separated by ,
- Supports:
 - Objects { }
 - Arrays []
- Does **NOT** support **functions or undefined**

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": 10021
  },
  "phoneNumber": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "fax",
      "number": "646 555-4567"
    }
  ]
}
```

JavaScript Object Notation

JSON supports ONLY:

- String
- Number
- Boolean
- Null
- Object
- Array

Stringify an object

- JSON is used to exchange data with servers
- When sending data to a server, the data has to be in string format
- Convert an object into a string with **JSON.stringify**

```
const obj = { name: "John", age: 30, city: "New York" };  
  
const json = JSON.stringify(obj);  
  
console.log(json);
```

Output

```
{"name":"John","age":30,"city":"New York"}
```

Parse JSON

- Data received from server is a string
- Convert JSON → object using:

```
const json = '{"name":"John","age":30,"city":"New York"}';  
  
const obj = JSON.parse(json);  
  
console.log(obj.name); // John
```

```
const json = '{"students":[{"name":"Ali"}, {"name":"Sara"}]}';  
  
const obj = JSON.parse(json);  
  
console.log(obj.students[0].name); // Ali
```

Accessing Data (Object from JSON)

- JSON data must first be converted into a JavaScript object
- Data is accessed using dot notation (.)

```
// Step 1: JSON (string)
const jsonData = '{"name":"Ali","age":20,"city":"Lahore"}';

// Step 2: Wrong access
console.log(jsonData.name); // undefined

// Step 3: Parse JSON → Object
const obj = JSON.parse(jsonData);

// Step 4: Access data
console.log(obj.name); // Ali

// Step 5: Modify object
obj.age = 25;
obj.country = "Pakistan";

// Step 6: Convert back to JSON
const updatedJSON = JSON.stringify(obj);

// Step 7: Output
console.log(updatedJSON);
```

Further Reading

W3Schools is a good and widely used resource to learn Javascript

- https://www.w3schools.com/js/js_variables.asp

- <https://javascript.info/types>

- <https://roadmap.sh/javascript>
